This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2023.1120000

LMGD: Log-Metric Combined Microservice Anomaly Detection through Graph-based Deep Learning

XU LIU[1], YUEWEN LIU[2], MIAOMIAO WEI[3], AND PENG XU[4],
[1]China Academy of Industrial Internet,Beijing (e-mail: liuxu@china-aii.com)
[2]State Key Lab of Networking and Switching Technology,Beijing University of Posts and Telecommunications,Beijing(e-mail: liuyw@bupt.edu.cn)
[3]State Key Lab of Networking and Switching Technology,Beijing University of Posts and Telecommunications,Beijing(e-mail: weimiaomiao@bupt.edu.cn)
[4]State Key Lab of Networking and Switching Technology,Beijing University of Posts and Telecommunications,Beijing(e-mail: xupeng@bupt.edu.cn)

Corresponding author: Yuewen Liu (e-mail: liuyw@bupt.edu.cn).

ABSTRACT Microservice architecture is a high-cohesion and low-coupling software architecture. Its core idea is to split the application into a set of microservices with a single function and independent deployment. Due to their complexity and large scale, microservice systems are typically fragile and failures are inevitable. Therefore, there is an urgent need for fast and accurate anomaly detection capabilities. However, the existing microservice anomaly detection methods do not pay attention to the multi-source data of the microservice system and thus have low accuracy. To address this limitation, we propose a Log-Metric Combined Microservice Anomaly Detection approach through Graph-based Deep Learning (termed as LMGD). First, we propose a time-aware LSTM prediction neural network to improve the accuracy of service dependency mining. Secondly, based on the service dependency graph, we propose an anomaly detection method based on log-metric fusion, which can more accurately describe the running status of microservices, thereby improving the accuracy of anomaly detection. The experimental outcomes demonstrate that compared with other state-of-the-art methods, our method improves recall and $F1$-score by 2.63% and 1.05%.

INDEX TERMS Anomaly Detection , Log , Metrics , Distributed Systems , GCN , GAT

I. INTRODUCTION

With the rapid advancement of internet technology, software systems have become increasingly complex, and traditional monolithic architectures are increasingly revealing their limitations in meeting modern software demands such as frequent deployment cycles, rapid feature iteration, and flexible horizontal scalability. In this context, microservices architecture has emerged as a novel design paradigm aimed at enhancing system flexibility and maintainability by decomposing applications into a series of small, independent services. These services are built around business capabilities and can be deployed, scaled, and upgraded independently [1]. Compared to traditional monolithic applications, microservices offer faster delivery speeds and higher scalability [2].

However, while microservices bring numerous benefits, they also introduce new challenges, particularly in system monitoring and anomaly detection. A typical microservices system comprises thousands of services distributed across a large number of servers, interacting frequently with one another. This high complexity and scale render microservices systems highly vulnerable, where a failure in any single service can trigger cascading failures throughout the entire system [3]. Additionally, due to the tight coupling between services, issues can rapidly propagate, affecting the overall stability of the system.Given these considerations, effective monitoring and anomaly detection within microservices systems becomes critical.

To date, many studies have been conducted on anomaly detection in microservice systems. The studies [4]–[7] mine the service dependency graph to establish a baseline model by learning the service logs of microservices running without faults, and then use the baseline model to detect whether the new logs deviate from the baseline model; the studies [8]–[10] monitor KPIs (Key Performance Indicators) on its services by installing agents and then processes such KPIs to detect anomalies. However, existing methods have the following limitation. Observable data in a microservice environment includes logs and metrics. Related studies only rely on a single observable data, which cannot fully discover whether the system is running abnormally and lead to low anomaly detection accuracy.

Therefore, we propose a Log-Metric Combined Microservice Anomaly Detection through Graph-based Deep Learning (LMGD). However, the following three challenges still remain:

1) **Incorporating Time Intervals in Service Dependency Mining:** Microservice logs exhibit varying time intervals, yet existing methodologies focus solely on the sequence of log entries, neglecting the critical aspect of time intervals between logs. Proper considera-

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

tion of these time intervals can enhance the precision of identifying dependencies among microservices.

2) **Integrating Metrics and Logs for Anomaly Detection:** Effective anomaly detection necessitates the joint analysis of multi-dimensional data, ensuring that no vital information is overlooked. However, the integration of metrics and logs is complicated by their inherent differences, such as distinct data structures and varying collection frequencies. Consequently, it is essential to develop a method that seamlessly merges these disparate data types into a unified analytical framework.
3) **Comprehensive Detection of Multiple Anomalies in Microservice Systems:** The inherent complexity of microservice architectures gives rise to various types of anomalies, including call and metric anomalies. Current detection methods often focus on a single type of anomaly, presenting a significant challenge in developing a detection method capable of identifying multiple anomalies concurrently.

In order to meet the above challenges, we have proposed the following solutions:

1) **Time-Aware Service Dependency Graph Mining:** To address the first challenge, we introduce a method for mining service dependencies that accounts for time intervals, thereby improving accuracy. Initially, template mining is applied to log data to extract time-series log sequences. These sequences are then fed into a time-aware Long Short-Term Memory (LSTM) neural network to predict potential successors for each node, ultimately generating the service dependency graph.
2) **Log-Metric Fusion for Enhanced Microservice State Learning:** In response to the second challenge, we propose a method for the fusion of log and metric data to gain a more comprehensive understanding of microservice states. Specifically, our approach integrates the service dependency graph with node metric information and utilizes a graph neural network to jointly learn the operational status of the system under historical normal conditions, including both node dependencies and states.
3) **Joint Rule-Based Anomaly Detection:** To tackle the third challenge, we propose an anomaly detection method based on joint rules, incorporating multiple breakpoints in the online detection phase to assess system abnormalities. We categorize anomalies into three types: redundant anomalies, sequence anomalies, and metric anomalies. The detection results are then stored as data labels in memory, a strategy that enhances both the recall and accuracy of anomaly detection.

In summary, our contributions are listed as follows:

1) To the best of our knowledge, this is the first attempt to study the fusion of logs and metrics for anomaly detection in microservice scenarios.
2) We propose a time-aware service dependency graph generation method to solve the problem of uneven log data intervals. We propose a method to integrate logs and metrics data, Which describe the dependencies between services and the running status of services with the aid of graph neural networks. We propose an anomaly detection method based on joint rules.
3) We validate our method using a real-world dataset and compare it with existing anomaly detection methods that rely on single data types, demonstrating the superiority of our approach. The results indicate that our method improves recall and $F1$-score by 2.63% and 1.05%, respectively, over state-of-the-art methods.

The rest of this paper is organized as follows. In Section 2, we summarize related work about anomaly detection methods for microservices and graph neural networks. In Section 3, we introduce our proposed algorithm LMGD. In Section 4, we conduct extensive experiments to illustrate the superiority of LMGD. In Section 5, we make the conclusion.

II. RELATED WORK

A. CALL CHAIN-BASED ANOMALY DETECTION

In the context of cloud-native data processing platforms, call chain-based anomaly detection can be categorized into two main types based on the source of call chain data: log-based call chain construction and monitoring tool-based call chain collection. In log-based call chain construction, the WFG-based method [11] detects call chain anomalies by learning a Workflow Graph (WFG) offline. The CFG-based method [5] detects anomalies by leveraging an offline-learned Control Flow Graph (CFG). The CPD-based method [12] performs anomaly detection by comparing the similarities between the control flow graphs of multiple call chains. DeepLog [13], on the other hand, utilizes an LSTM model to detect call path anomalies within the call chain. This method trains independent LSTM models for each node in the call chain to detect response time anomalies. Compared to log-based methods, monitoring tool-based call chain collection offers better real-time performance, enabling timely detection of platform performance issues. The AEVB [14] method proposes an unsupervised deep Bayesian network model to detect response time anomalies in call chains. Seer [15] uses a CNN to perform dimensionality reduction on data and then employs LSTM to learn temporal and spatial patterns in call chain data. This method effectively analyzes and predicts QoS, providing crucial insights and support for system performance improvement. The Multimodal LSTM [14] model simultaneously learns and detects call path anomalies and response time anomalies in call chains. TraceAnomaly [16] and CRISP [17] represent call chains as Service Tracking Vectors (STV) and use Variational Autoencoders (VAE) [18] to detect both call path anomalies and response time anomalies.

B. LOG-BASED ANOMALY DETECTION

In log-based anomaly detection research, unsupervised learning methods are commonly employed. These methods model the logs recorded during the stable operation of the platform using unsupervised learning algorithms and then use this

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

model to detect whether newly recorded log events match the model to determine if the platform is experiencing anomalies. The OASIS [5] method analyzes logs during stable operation to construct a control flow graph that models the platform's behavior under normal conditions, identifying which events should be recorded by which service and in what order. Jia et al. [19] and LogSed et al. [20] analyze logs during stable operation to mine a time-weighted control flow graph for each service to model its internal processes.

C. KPI-BASED ANOMALY DETECTION

In KPI-based anomaly detection research, abnormal situations are identified by monitoring key performance indicators (KPIs) of nodes in the platform, such as CPU usage, memory usage, and response time. Common methods include both unsupervised and supervised learning. In unsupervised learning, the unsupervised learning algorithm processes KPIs detected during the platform's normal operation to form a model, which is then used to detect if new KPIs are abnormal. Gulenko et al. [10] established different models for different services within the platform, modeling KPIs as vectors and clustering these vectors using clustering algorithms. LOUD et al. [21] paired each KPI with the corresponding service and trained a model for the entire platform. By establishing KPI models and causal graphs, KPIs that fall outside the acceptable range of the model are identified as anomalies. In supervised learning, ADS [22] and PreMiSE [23] injected specific faults into specific services during the training phase and labeled the KPIs to distinguish between KPIs collected under normal conditions and those collected during faults.

III. PROBLEM DEFINITION

In this section, we will introduce necessary notations and formally formulate our problem. We define the raw data set as Rdata. Among them, the original log set is L={$log_1, log_2, \ldots, log_n$}, after parsing by the log template parser, a structured log set can be obtained, expressed as D={$d_1, d_2, \ldots, d_n$ } and the log template set is K={$k_1, k_2, ..., k_m$}. d_i is the structured log corresponding to log_i, and the structured log can be defined by a triple (k, pod, t_s), where k is the log template, pod is the label of the microservice node to which the log belongs, and t_s is the timestamp when the log is printed.The output of subsequent prediction stages is vector V_0. The subsequent prediction result of real-time detection is V_i. The input of the graph attention network is a set of node features h={$\vec{h_1}, \vec{h_2}, \ldots, \vec{h_N}, \vec{h_i} \in R$}. N is the number of nodes, F is the number of features of a node. We categorize the log data into two types: normal and abnormal. Among the abnormal samples, we further classify them into three levels of severity based on the degree of abnormality: mild, moderate, and severe.

IV. LMGD OVERVIEW

Figure 1 provides a comprehensive overview of our proposed method, LMGD. The input to LMGD consists of microservice transaction logs and the associated metrics of each service node, while the output is a determination of whether the analyzed data batch is abnormal. The approach is divided into two primary stages: the offline stage, which focuses on constructing service dependency graphs from logs and training a baseline model that reflects the system's normal operational state, and the online stage, which monitors deviations between real-time data and the established baseline model.

In the offline phase, two key processes are undertaken. First, microservice transaction logs are analyzed to extract information necessary for constructing an inter-service dependency graph, which elucidates the interactions among components within the microservice architecture. Second, using the extracted log data, a baseline model is trained to represent the normal behavior of the system. This model serves as the foundation for anomaly detection during the subsequent online phase.

The online phase is concerned with real-time surveillance of microservice behavior. By comparing the current data flow against the baseline model established in the offline phase, the system identifies potential anomalies. Significant deviations from the baseline are flagged as potential anomalies, facilitating timely detection and resolution of issues within the microservice architecture. This real-time monitoring capability enhances the stability and reliability of the system.

LMGD comprises three core components: the generation of the service dependency graph, data fusion based on graph neural networks, and anomaly detection. Section 4.1 details the generation of the service dependency graph, where log files are processed into template sequences and a time-aware deep learning model is employed to extract service dependencies. Section 4.2 discusses the data fusion process, in which the microservice system is modeled as a graph, and a two-layer graph neural network is utilized to train a baseline model representing normal system states. Finally, Section 4.3 addresses anomaly detection, where joint conditions are designed to systematically identify anomalies within the system.

A. GENERATION OF SERVICE DEPENDENCY GRAPH

First, we preprocess the raw log data and parse the log text into structured data. The template extraction is performed on the logs, and different types of logs are distinguished by clustering. The log parsing steps are shown in Figure 2. Drain is an online log parsing method based on a fixed depth tree. When a new raw log message is sent, Drain will preprocess the log data using simplified regular expressions based on knowledge of the domain to which the log belongs, so as to better match the specific design rules encoded in the internal nodes of the tree. If the log can be matched to an appropriate log group, the log will be grouped with the previously collected log group. If there is no matching result, a new log group will be created based on the log information.

Next, we will obtain the log template sequence, which is ready for subsequent service dependency graph mining. Logs with time tags, node tags, and template tags are obtained through log parsing, and the logs collected by each node are sorted by time to obtain a log sequence. This sequence

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

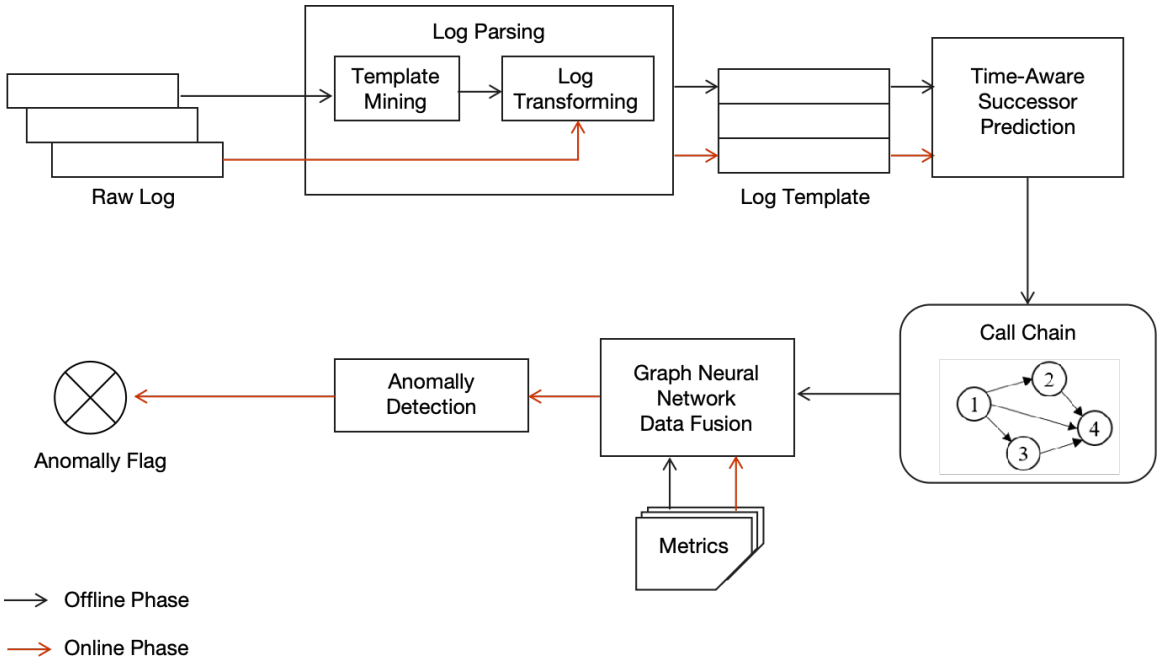


FIGURE 1. Approach Overview

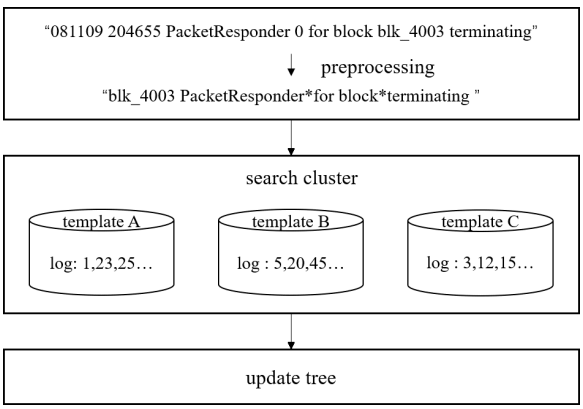


FIGURE 2. Log Parsing Algorithm Process

contains the possible calls of each node. For example, node c always follows After node b, it can be judged that the possibility of b calling c is very high, that is, the successor node of b node is very likely to be c node.

Since each node of the system executes tasks with multiple threads, the generated log sequence is chaotic and interlaced, so a neural network with a certain depth can better detect the frequent patterns among them. Input this sequence into the LSTM artificial neural network prediction model with time perception ability, learn the frequent pattern of node calls, and mine the statistically significant successor nodes of each node template. Here, the LSTM multi-input single-output prediction model is used to predict possible successor nodes for each node.

Meanwhile, the standard LSTM model only implicitly deals with uniform time intervals in the input sequence, so the LSTM unit is equipped with a time-aware mechanism to pay attention to the impact of different time intervals on the prediction results. It is well understood that sequences with short log intervals are more likely to contain actual calls than sequences with longer intervals, the latter cell's contribution will decrease as the interval increases.

Add Δt information to the input gate and forget gate to pay attention to the impact of different time intervals:

$$f_t' = e^{-\Delta t} f_t. \tag{1}$$

$$i_t' = (1 - e^{-\Delta t}) i_t. \tag{2}$$

where Δt is the time interval, representing the time difference between two log events. It measures the impact of the time interval on the prediction results; $f_t$is the forget gate in the LSTM unit. The forget gate controls the extent to which past information is forgotten. In the original formula, it is the output of the forget gate. In the new formula, it is adjusted by Δt to generate f_t'; i_t is the input gate in the LSTM unit. The input gate controls the extent to which new information is added to the LSTM unit state. In the original formula, i_t is the output of the input gate. In the new formula, it is adjusted by Δt to generate i_t'; f_t' is the adjusted forget gate output. $f_t' = e^{-\Delta t} f_t$ indicates that the forget gate output is influenced by the time interval Δt. As Δt increases, f_t' decreases, meaning the degree of forgetting past information is higher; i_t' is the adjusted input gate output. $i_t' = (1 - e^{-\Delta t}) i_t$ indicates that the input gate output is influenced by the time interval Δt. As Δt increases, i_t' increases, meaning the degree of incorporating new information is higher.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

The prediction model consists of three layers: the input layer, the neural network hidden layer, and the output layer. The input layer encodes all logs into one-hot vectors. The input layer receives the log template sliding window w=<x_{t-h}, $\ldots$, x_{t-2}, x_{t-1}> of the training data set in the training phase, and receives the log template sliding of the detection data set in the detection phase window w. where x represents the log template in the sliding window. h represents the length of the sliding window, the number of past log templates considered in each prediction.The output layer is responsible for converting the hidden state set H_{t-h}, $\ldots$, H_{t-2}, H_{t-1} output by the neural network layer into the subsequent prediction probability vector V_w of the final output log template.where H represents the hidden state in the neural network. The hidden state H_t captures the information from the input sequence up to time t. It is an internal state used by the LSTM to store information about previous inputs in the sequence.

In the detection phase, when the model predicts a large difference between the log template and the actual log template, exceeding the set threshold, it is determined that the log has an abnormal call.

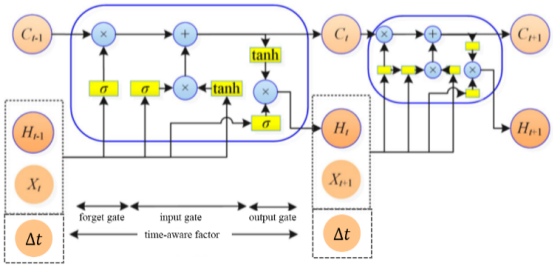


FIGURE 3. Time-aware LSTM Algorithm

Finally, we obtain the possible successors and separate possibilities of each node V_0. Its data structure is a two-dimensional matrix, and the rank of the matrix is the number of nodes n. Among them, "0.35" means that the probability that node A calls node B is 35%.

$$\begin{pmatrix} 0.01 & 0.35 & \ldots & 0.02 \\ 0.12 & 0.00 & \ldots & 0.01 \\ \ldots & \ldots & \ldots & \ldots \\ 0.00 & 0.00 & \ldots & 0.05 \end{pmatrix} \tag{3}$$

Among them, the call from node to node whose probability is lower than a threshold is regarded as an error, so as to prune the data and reduce the cost of subsequent graph neural network information transmission and anomaly detection. In the generated graph, each point represents a service node, an edge represents the existence of calls between the two services, and the weight of the edge represents the degree of dependence between services.

B. DATA FUSION BASED ON GRAPH NEURAL NETWORK

Based on the service dependency graph obtained in the previous stage, we integrate it with metrics to jointly learn the running status of microservices. The microservice running status information contained in the log information has been fully mined in the service dependency graph generation stage. In addition, the metrics data also contains rich information. Therefore, we propose a two-layer graph neural network to fuse the service dependency graph and metrics to characterize microservice systems.

First, we propose a graph structure to describe the microservice. It is known that the observable metrics dimensions of all service nodes are the same, and assuming that the contribution of the same metrics on different nodes is the same, at this time, the microservice scenario can be composed of a large graph of service nodes and each node and the metrics is composed of tree subgraphs. In the large graph, nodes represent microservices, and edges between points represent dependencies between microservices, that is, invocation situations. In the subgraph, there are only central nodes and edge nodes. The central node represents microservices, and the edge nodes are service-level observable metrics. There are no edges between metrics. There are only dependencies between central nodes and edge nodes in the subgraph, and the weight of edge nodes, the contribution degree, is different, and the structure is shown in Figure 4.

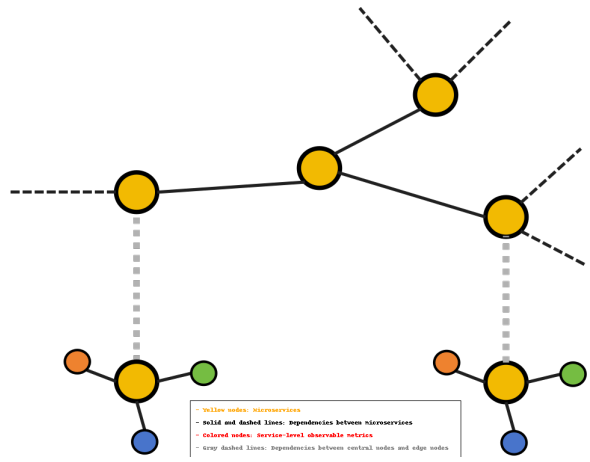


FIGURE 4. Service Dependency Graph and Subgraph

Then, we apply graph neural networks to graph structures to capture the features of a node by aggregating and transforming the features of its neighbors, thus taking into account the influence of related nodes on nodes. For subgraphs, the connectivity between known nodes is certain, but the metrics contribution weight is unknown, the graph attention network (Graph Attention Network, GAT) can learn the attention weight, and the operation of the self-attention layer can be in all edges are performed in parallel, and the calculation of output features of all nodes is also performed in parallel. The attention mechanism can be shared and applied to all edges in the graph. Therefore, the graph attention network is selected to learn the weight information of metrics nodes and service nodes.

However, the information of each edge of the large graph between services is certain, and the edges are weighted and directed, so there is no need to pay attention to the learning

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

of the edges. After the GAT training of the sub-graph is completed, the characteristic data of the service nodes can be obtained. At this time, it is necessary to focus on how much each node is affected by neighbor nodes, that is, nodes with calling relationships. Here, a graph convolutional neural network (Graph Convolutional Network, GCN) is used to process neighborhood information. Each convolutional layer can process a The information transfer of multi-order neighborhoods can be realized by stacking several convolutional layers. Microservice nodes are not only affected by neighbor nodes, but the nodes involved in the entire call chain may affect the operation of the service. Therefore, the design of the number of convolutional layers will be discussed later.

The input of the graph attention network is a set of node features $h=\{\vec{h_1}, \vec{h_2}, \ldots, \vec{h_N}, \vec{h_i} \in R^F\}$, N is the number of nodes, F is the number of features of a node, here is 1. Apply the weight matrix $W \in R$, which is a shared linear transformation, to each node, perform self-attention on the node, and calculate the attention coefficient:

$$e_{ij} = \alpha(W\vec{h_i}, W\vec{h_j}). \tag{4}$$

Here, e_{ij} represents the importance of node j to node i. Then calculate the attention coefficient between nodes, the formula α is as follows:

$$\alpha_{ij} = \frac{exp(LeakyRelu(\vec{a}^T[W\vec{h_i}][W\vec{h_j}]))}{\sum_{k \in N_i} exp(LeakyRelu(\vec{a}^T[W\vec{h_i}][W\vec{h_k}]))}. \tag{5}$$

After obtaining the normalized attention coefficient, calculate the updated output features,and σ is the activation function

$$\vec{h_i'} = \sigma(\sum_{j \epsilon N_i} \alpha_{ij} W\vec{h_j}). \tag{6}$$

For subsequent large-scale training, the output features of the previous layer are used as input to the graph convolutional neural network, and finally, the results are probabilized through the Softmax activation function to obtain the probability distribution of the predicted results:

$$Z = softmax(\hat{A}ReLU(\hat{A}XW^{(0)}W^{(1)})). \tag{7}$$

where Z is the final output result, which is the probability distribution of the prediction results after applying the softmax activation function.$\hat{A}$ is the normalized adjacency matrix of the graph, containing the information about the edges between nodes. X is the initial node feature matrix, where each row corresponds to the feature vector of a node.$W^{(0)}$ and $W^{(1)}$ is the weight matrices used in the training process for linear transformation of node features.

After the entire network, the feature of each node of GCN has changed from X to Z. During the whole process, the adjacency matrix A is shared and unchanged.The layer-to-layer propagation method is as follows, and H is the feature of node,$\tilde{D}$ is the degree matrix of the normalized adjacency matrix $\tilde{A}$:

$$H^{l+1} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}). \tag{8}$$

It is worth noting that this graph neural network information transfer method is not only applicable to the service dependency graph generated based on the log in this paper, but also applicable to the call chain generated by the trace collected by link tracing and other methods.

C. ANOMALY DETECTION

The core concept behind anomaly detection is to compare real-time logs and metrics against log templates, service dependency graphs, and graph neural networks derived from normal system behavior during the training phase. By doing so, the system can diagnose potential anomalies. During training, various techniques are employed to model the system's normal baseline state, allowing for the establishment of multiple checkpoints to detect anomalies, a process referred to as the joint rule-based anomaly detection method. This method is typically divided into three components: (1) determining whether a log can be parsed and mapped; if it cannot, it is classified as a redundancy anomaly; (2) assessing whether there is an anomaly within the log sequence; if one is detected, it is considered a sequence anomaly; and (3) evaluating the deviation between the graph neural network, the service dependency graph, and the metrics from the training phase; significant deviations indicate a metrics anomaly.

First, we verify whether the real-time log data can be mapped to a predefined log template. If the log cannot be classified into any existing log group, it is considered an anomalous log. This is intuitive, as the system may generate abnormal logs under fault conditions. The logs used during the parsing stage are all collected under normal system operation. Consequently, any logs that cannot be clustered with existing log groups—obtained through clustering during normal operation—are flagged as abnormal.

Next, we examine whether the log sequence exhibits any anomalies. Successfully mapping a log to its corresponding template does not guarantee that the system is functioning normally, as anomalies can manifest in various forms. Even if a service failure does not produce an error log, it may still disrupt multiple services, impacting their invocation sequences. At this stage, attention is focused on the real-time log template sequence, which is then compared with the time-aware successor prediction data to determine whether an exception has occurred. The time-aware successor prediction stage provides the potential successor nodes for each service, and any deviation from this expected sequence is flagged as anomalous.

Finally, we evaluate the metrics data for any irregularities. Anomalies may also be reflected in changes to system metrics, such as a sudden spike in resource usage. The real-time metrics data is fed into the graph neural network, which is followed by a fully connected layer to determine whether the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

metrics indicate an anomaly. By applying this joint rule-based anomaly detection method, which encompasses redundancy anomalies, sequence anomalies, and metrics anomalies, the system can comprehensively determine whether it is functioning abnormally.

V. EXPERIMENTS

A. DATASET

This paper adopts the multi-source distributed system dataset proposed by literature [24] for artificial intelligence analysis. This dataset captures high-quality multi-source data consisting of distributed traces, application logs, and metrics for distributed systems. Remove the call information from the trace data and use it. The actual call information is used to verify the accuracy of the results in the anomaly detection phase.

The dataset comes from 5 different nodes of the distributed system, including 829,562 log data, each node has an average of 239,127 metrics, and a total of 1,195,637 metrics. Among them, the injected fault data accounts for about 6.8%, and this dataset runs through the entire experimental study.

B. BASELINE

The following four state-of-the-art log-based or trace-based anomaly detection methods are used as baselines.

1) DeepTraLog [25] uses a unified graph representation to describe the complex structure of the trace and the log events embedded in the structure. Based on the graph representation, DeepTraLog trains a deep SVDD model based on GGNNs by combining traces and logs, and detects new traces and anomalies in the corresponding logs.

2) TraceAnomaly [16] is a trace anomaly detection method that only considers service-level traces (operations are not considered). It employs a posterior flow-based variational autoencoder (VAE) to detect anomalous trajectories.

3) DeepLog [13] is a log anomaly detection method that treats logs as a sequence of events. It employs LSTM models to predict the next logging event in the sequence and identify possible anomalies.

4) LogAnomaly [26] is a log anomaly detection method that treats the log as a sequence of events and considers the count of distinct log events as an additional feature. It employs LSTM models to learn sequential and quantitative patterns. Similar to DeepLog, it detects anomalies by predicting the next log event.

Since TraceAnomaly only considers trace data, when using them, only provide trace to them. DeepLog only considers log data and treats it as a sequence of events. Therefore, combine all events of different services in a trace into a sequence of events sorted by timestamp, and provide the sequence of events for all traces as input to these two methods. TraceAnomaly and DeepLog provide open source code implementations, use them directly. DeepTraLog uses trace data and log data.

TABLE 1. Method effectiveness evaluation results

Methods	Precision	Recall	F1-Score
DeepTraLog	0.830	0.876	0.852
TraceAnomaly	0.742	0.205	0.321
DeepLog	0.608	0.848	0.741
LogAnomaly	0.415	0.877	0.582
LMGD	**0.827**	**0.902**	**0.862**

C. METRICS

Use precision, recall and $F1$ score to measure the effectiveness of anomaly detection.

Accuracy rate: the percentage of real anomalies in all detected anomalies, which can be understood as the precision rate, expressed as:

$$Precision = \frac{TP}{TP + FP}. \tag{9}$$

Recall rate: the percentage of detected abnormalities in all abnormal data, which can be understood as recall rate, expressed as:

$$Recall = \frac{TP}{TP + FN}. \tag{10}$$

$F1$-Score: the harmonic mean of precision and recall, represented as

$$F1 = \frac{2Precision \cdot Recall}{Precision + Recall}. \tag{11}$$

Accuracy is an important metric to measure the accuracy of outlier recognition, and wrong anomaly recognition will affect the accuracy of the algorithm. The recall rate reflects the completeness of outlier identification, and the lack of anomaly identification will affect the recall rate of the algorithm. The $F1$ score is the weighted harmonic average of Precision and Recall, which can better evaluate the results of anomaly detection.

D. EXPERIMENTAL RESULTS

Table 1 shows the effectiveness evaluation results of different methods. The results show that LMGD outperforms the baseline methods and achieves 0.827 precision, 0.902 recall and 0.862 $F1$-score.

The precision and recall of the trace-based method TraceAnomaly are low. This approach does not take logs into account, so anomalies in log events cannot be detected. The input to TraceAnomaly is sequence-based trace data, with a special focus on response time, so it can only detect anomalies where the sequence of spans or the distribution of response times have a significant impact.

The log-based methods DeepLog and LogAnomaly have high recall and low precision. These two methods take into account the events within the span and thus can detect anomalies in the sequence of spans and the sequence of log events. However, this method relies on predicting the next event in a

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

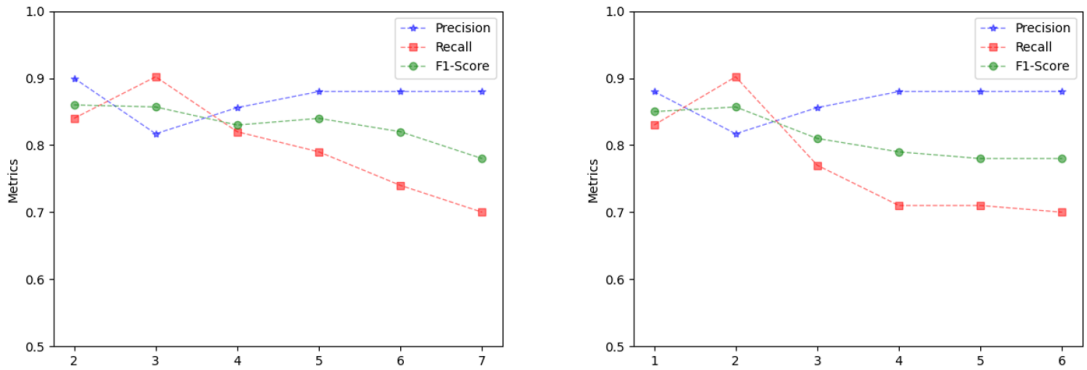


FIGURE 5. Accuracy Metrics in different Hidden Layers in GCN/GAT

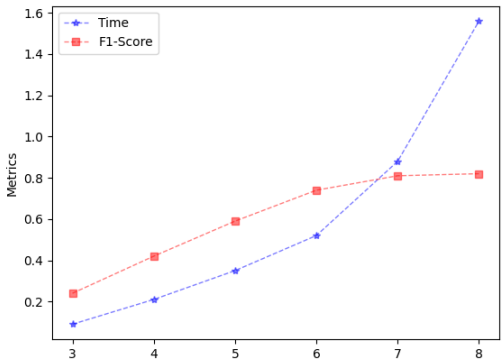


FIGURE 6. Performance Metrics in different Maximum leaf nodes

sliding window of the event sequence, and thus has no structured global view. As a result, subsequences of events that are not visible may be incorrectly reported as exceptions. These two log anomaly detection methods are usually performed on a single sequence of log events such as HDFS datasets. Therefore, it does not work well for microservice anomaly detection involving parallel and asynchronous service calls involving multiple service instances and distributed log events.

DeepTraLog has high precision and high recall, it considers all span events and log events of a call chain, and has better performance than log anomaly detection methods based on sliding window log analysis. DeepTraLog is the only baseline method that considers multi-source trace and log data for fusion analysis, so it shows better performance. In term of precision, DeepTraLog is slightly higher than LMGD, and it is easy to understand that trace data can provide more accurate system global information than log data, which is obtained at the expense of system intrusion and performance.

LMGD has the highest $F1$ score compared to the four baseline methods, thus demonstrating its excellent performance. The precision rate is slightly lower than DeepTraLog, and the recall rate is the highest. Based on the detailed analysis of the previous algorithm, it can be seen that the anomaly detection algorithm of LMGD pays attention to the occurrence of three kinds of anomalies, so it can check all the anomalies; under the requirements of the scene, this method mines the information in the log and metric data as much as possible to obtain relatively high precision and highest overall $F1$ score.

In summary, LMGD is effective in microservice anomaly detection, achieving an average improvement of 2.63% and 1.05% in recall and $F1$-score than existing trace-based and log-based anomaly detection methods, respectively.

E. ABLATION EXPERIMENT

The numbers of hidden layers of GCN and GAT are two important parameters of this algorithm. The number of hidden layers of GCN determines the number of iterations of message passing between nodes, which has a greater impact on the underlying representation of the graph. The number of hidden layers of GCN is usually set to 2 to 4. **The results show that both recall and $F1$ score decrease as the number of hidden layers increases.** This is usually because more hidden layers can easily lead to over-smoothing, making features indistinguishable and thus affecting classification accuracy. In addition, the system contains five nodes with low complexity. Excessive data exchange will lose the characteristics of the data, and it will be difficult to mine the dependencies between services.

The number of hidden layers of GAT determines the number of iterations of node and metrics message transmission, which has a greater impact on the judgment of metrics weight. The influence of the number of hidden layers of GAT on the accuracy of anomaly detection is shown in the figure 5. **The experimental results show that the 2-layer graph network can achieve better training results.** According to the characteristics of the data set, it is also easy to understand, and the metrics dimensions used are fewer, only three, and the complexity is very low. Excessive data exchange will lose the characteristics of the data, and it is difficult to mine the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

metrics trend.

At the same time, the system also analyzes the impact of the maximum number of leaf nodes in the tree structure in the template extraction stage on the performance of the algorithm. The difference of the largest leaf node will have an important impact on the time complexity of the algorithm and the effect of anomaly detection. **Figure 6 shows that with the increase of the largest leaf node, the time consumption of the algorithm increases exponentially, and the growth rate of the $F1$ score gradually slows down.** This is because, if there is no limit to the maximum leaf node, the number of log groups will increase after the template is extracted, the matching time of log data will increase, and the number of nodes in the prediction stage of subsequent nodes will also increase, and the training time will naturally increase to a large extent. From the perspective of abnormal detection effect, the increase of the largest leaf node, the accuracy of template extraction and log matching will be improved, and the learning of the system state will be more accurate, so the $F1$ score will increase, but after reaching the critical point, the $F1$ score will tend to be stable. Because subsequent fetches and matches will no longer change. In this case, this paper selects a critical point to reduce the training time of the algorithm on the premise of ensuring the effectiveness of anomaly detection.

VI. CONCLUSION

In this paper, we propose an anomaly detection method in a microservice environment. This method solves the problem of not being able to pay attention to the time interval of log data, solves the problem of log metric data fusion, and also solves the problem of simultaneous detection of multiple anomaly types. We propose a Log-Metric Combined Microservice Anomaly Detection through Graph-based Deep Learning (LMGD). The method is divided into three parts:

(1) To address the issue of irregular log data intervals, we propose a time-aware service dependency graph generation method.

(2) we propose a novel approach for integrating log and metrics data to capture the inter-service dependencies and operational status of services, leveraging the capabilities of graph neural networks.

(3) We propose an anomaly detection method that is based on conjoint rule analysis. We conduct lots of experiments on the dataset proposed by Sasho Nedelkoski and compare our method with some current baselines in the field of anomaly detection. The experimental results indicate that the proposed method significantly outperforms state-of-the-art approaches, with recall and F1-score metrics reaching 0.902 and 0.862, respectively.

In our future work, we will study the influence of more parameters on the algorithm effect, and make more efforts in reducing the algorithm complexity.

REFERENCES

[1] Florian Auer, Valentina Lenarduzzi, Michael Felderer, and Davide Taibi. From monolithic systems to microservices: An assessment framework. *Information and Software Technology*, 137:106600, 2021.

[2] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, et al. Practical root cause localization for microservice systems via trace analysis. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, pages 1–10. IEEE, 2021.

[3] Lingzhi Wang, Nengwen Zhao, Junjie Chen, Pinnong Li, Wenchi Zhang, and Kaixin Sui. Root-cause metric location for microservice systems via log anomaly detection. In *2020 IEEE International Conference on Web Services (ICWS)*, pages 142–150. IEEE, 2020.

[4] Wiliam Setiawan, Yohen Thounaojam, and Apurva Narayan. Gwad: Greedy workflow graph anomaly detection framework for system traces. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 2790–2796. IEEE, 2020.

[5] Animesh Nandi, Atri Mandal, Shubham Atreja, Gargi B Dasgupta, and Subhrajit Bhattacharya. Anomaly detection using program control flow graph mining from execution logs. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 215–224, 2016.

[6] Tong Jia, Lin Yang, Pengfei Chen, Ying Li, Fanjing Meng, and Jingmin Xu. Logsed: Anomaly diagnosis through mining time-weighted control flow graph in logs. In *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*, pages 447–455. IEEE, 2017.

[7] Pooja Aggarwal, Ajay Gupta, Prateeti Mohapatra, Seema Nagar, Atri Mandal, Qing Wang, and Amit Paradkar. Localization of operational faults in cloud applications by mining causal dependencies in logs using golden signals. In *International Conference on Service-Oriented Computing*, pages 137–149. Springer, 2020.

[8] Li Wu, Jasmin Bogatinovski, Sasho Nedelkoski, Johan Tordsson, and Odej Kao. Performance diagnosis in cloud microservices using deep learning. In *International Conference on Service-Oriented Computing*, pages 85–96. Springer, 2020.

[9] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. Microrca: Root cause localization of performance issues in microservices. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9. IEEE, 2020.

[10] Anton Gulenko, Florian Schmidt, Alexander Acker, et al. Detecting anomalous behavior of black-box services modeled with distance-based online clustering. In *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, pages 912–915. IEEE, 2018.

[11] Qiang Fu, Jianguo G Lou, Yi Wang, et al. Execution anomaly detection in distributed systems through unstructured log analysis. In *2009 Ninth IEEE International Conference on Data Mining*, pages 149–158. IEEE, 2009.

[12] Lingfang Bao, Qinghua Li, Peng Lu, et al. Execution anomaly detection in large-scale systems through console log analysis. *Journal of Systems and Software*, 143:172–186, 2018.

[13] Min Du, Feifei Li, Guineng Zheng, et al. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298. ACM, 2017.

[14] Sasho Nedelkoski, Jorge Cardoso, and Odej Kao. Anomaly detection from system tracing data using multimodal deep learning. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, pages 179–186. IEEE, 2019.

[15] Yuxiong Gan, Ying Zhang, Kaushik Hu, et al. Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 19–33. ACM, 2019.

[16] Peng Liu, Haowen Xu, Qianyu Ouyang, et al. Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pages 48–58. IEEE, 2020.

[17] Zhiyuan Zhang, Murali K Ramanathan, Prashant Raj, et al. CRISP: Critical path analysis of Large-Scale microservice architectures. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 655–672, 2022.

[18] Myunghwan Kim, Roshan Sumbaly, and Sam Shah. Root cause detection in a service-oriented architecture. *ACM SIGMETRICS Performance Evaluation Review*, 41(1):93–104, 2013.

[19] Tong Jia, Pengfei Chen, Lin Yang, et al. An approach for anomaly diagnosis based on hybrid graph model with logs for distributed services. In *2017*

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2024.3481676

IEEE International Conference on Web Services (ICWS), pages 25–32. IEEE, 2017.

[20] Tong Jia, Lin Yang, Pengfei Chen, et al. Anomaly diagnosis through mining time-weighted control flow graph in logs. In *Proceedings of the 2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*, pages 25–30. IEEE, 2017.

[21] Leonardo Mariani, Claudio Monni, Mauro Pezzé, et al. Localizing faults in cloud systems. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, pages 262–273. IEEE, 2018.

[22] Qijun Du, Tao Xie, Yang He, et al. Anomaly detection and diagnosis for container-based microservices with performance monitoring. In *Algorithms and Architectures for Parallel Processing: 18th International Conference, ICA3PP 2018, Guangzhou, China, November 15-17, 2018, Proceedings, Part IV*, pages 560–572. Springer, 2018.

[23] Leonardo Mariani, Mauro Pezzé, Omar Riganelli, et al. Predicting failures in multi-tier distributed systems. *Journal of Systems and Software*, 161:110464, 2020.

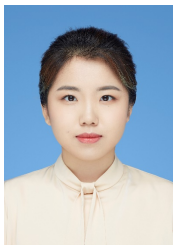
[24] Sasho Nedelkoski, Jasmin Bogatinovski, Ajay Kumar Mandapati, Soeren Becker, Jorge Cardoso, and Odej Kao. Multi-source distributed system data for ai-powered analytics. In *Service-Oriented and Cloud Computing: 8th IFIP WG 2.14 European Conference, ESOCC 2020, Heraklion, Crete, Greece, September 28–30, 2020, Proceedings 8*, pages 161–176. Springer, 2020.

[25] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th International Conference on Software Engineering*, pages 623–634, 2022.

[26] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *IJCAI*, volume 19, pages 4739–4745, 2019.

XU LIU received the Ph.D. Degree in Computer Science from Beijing University of Posts and Telecommunications, China in 2018. He is currently a Senior engineer of China Academy of Industrial Internet. His research interests include industrial internet and big data.

YUEWEN LIU is a master in Computer Science and Technology from Beijing University of Posts and Telecommunications. His research areas are machine learning and AI native



MIAOMIAO WEI is a master in Computer Science and Technology from Beijing University of Posts and Telecommunications. Her research areas are machine learning and anomly detection

PENG XU received the Ph.D. Degree in Communication and Information Systems from Beijing University of Posts and Telecommunications, China in 2004. He is currently an associate professor of Beijing University of Posts and Telecommunications. His research interests include AIOps and Industrial Big Data.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4